Formulaires et champs de saisie en SwiftUI

420-249-AH - Programmation mobile iOS

1. Pourquoi utiliser un formulaire ?

Une application mobile doit souvent permettre à l'utilisateur de saisir des informations.

Exemples :

- Créer un compte
- Modifier un profil
- Ajouter un ticket
- Ajouter un produit
- Créer une réservation

SwiftUI fournit plusieurs contrôles adaptés à la saisie.

2. Principaux contrôles de saisie

Contrôle	Usage
`TextField`	Texte court sur une ligne
`SecureField`	Information sensible, comme un mot de passe
`TextEditor`	Texte long sur plusieurs lignes
`Form`	Conteneur structuré pour un formulaire
`Section`	Regroupement de champs dans un formulaire

Ces contrôles fonctionnent avec une variable @State ou avec une donnée provenant d'un ViewModel.

3. TextField

TextField permet de saisir du texte sur une seule ligne. Le premier paramètre est le texte indicatif affiché lorsque le champ est vide.

```
struct ProfilView: View {
    @State private var nom = ""

    var body: some View {
        TextField(
            "Nom",
            text: $nom
        )
    }
}
```

Le $ transmet un Binding<String>, comme vu avec la gestion de l'état : le champ peut lire et modifier nom. Le texte saisi est donc automatiquement conservé dans nom.

4. Ajouter un style

```
TextField(
    "Nom",
    text: $nom
)
.textFieldStyle(.roundedBorder)
```

Cela donne un champ visuellement plus identifiable.

5. Exemple - Formulaire d'inscription

```
struct InscriptionView: View {
    @State private var nom = ""
    @State private var courriel = ""

    var body: some View {
        VStack(spacing: 16) {
            TextField(
                "Nom",
                text: $nom
            )
            TextField(
                "Courriel",
                text: $courriel
            )
        }
        .textFieldStyle(.roundedBorder)
        .padding()
    }
}
```

Le modificateur .textFieldStyle appliqué au VStack s'applique aux deux champs.

6. Adapter le clavier

Le clavier affiché doit correspondre au type d'information saisie :

Besoin	Modificateur
Courriel	`.keyboardType(.emailAddress)`
Téléphone	`.keyboardType(.phonePad)`
Nombre entier	`.keyboardType(.numberPad)`
Nombre décimal	`.keyboardType(.decimalPad)`

Pour un courriel, on ajoute aussi :

```
TextField(
    "Courriel",
    text: $courriel
)
.keyboardType(.emailAddress)
.textInputAutocapitalization(.never)
.autocorrectionDisabled()
```

- .textInputAutocapitalization(.never) : pas de majuscule automatique
- .autocorrectionDisabled() : pas de correction automatique

C'est plus approprié pour une adresse courriel.

7. SecureField

Pour une information sensible comme un mot de passe :

```
SecureField(
    "Mot de passe",
    text: $motDePasse
)
```

Le texte saisi n'est pas affiché directement : il est remplacé par des points.

8. Exemple - Écran de connexion

```
struct ConnexionView: View {
    @State private var courriel = ""
    @State private var motDePasse = ""

    var body: some View {
        VStack(spacing: 16) {
            TextField(
                "Courriel",
                text: $courriel
            )
            .keyboardType(.emailAddress)
            .textInputAutocapitalization(.never)

            SecureField(
                "Mot de passe",
                text: $motDePasse
            )

            Button("Connexion") {
            }
        }
        .padding()
    }
}
```

9. Afficher ou masquer le mot de passe

Pour permettre d'afficher le mot de passe, on utilise un état et on choisit entre TextField et SecureField :

```
struct MotDePasseView: View {
    @State private var motDePasse = ""
    @State private var afficherMotDePasse = false

    var body: some View {
        HStack {
            if afficherMotDePasse {
                TextField(
                    "Mot de passe",
                    text: $motDePasse
                )
            } else {
                SecureField(
                    "Mot de passe",
                    text: $motDePasse
                )
            }

            Button {
                afficherMotDePasse.toggle()
            } label: {
                Image(
                    systemName: afficherMotDePasse ? "eye.slash" : "eye"
                )
            }
        }
    }
}
```

- afficherMotDePasse indique si le texte est visible
- Le bouton inverse l'état avec toggle()
- L'icône change selon l'état : eye ou eye.slash

10. TextEditor

TextField est surtout destiné à une ligne de texte. Pour un texte plus long, on utilise TextEditor :

```
@State private var description = ""

var body: some View {
    TextEditor(
        text: $description
    )
}
```

TextEditor n'a pas de texte indicatif : on ajoute généralement un Text au-dessus.

11. Exemple - Description d'un ticket

```
VStack(
    alignment: .leading
) {
    Text("Description")
        .font(.headline)
    TextEditor(
        text: $description
    )
    .frame(minHeight: 120)
}
```

Cela convient bien à :

- La description d'un problème
- Un commentaire
- Des notes
- Un message

12. TextField ou TextEditor ?

Critère	TextField	TextEditor
Longueur	Texte court	Texte long
Lignes	Une ligne	Plusieurs lignes
Exemples	Nom, courriel, titre	Description, commentaire, notes

13. Form

Form crée des interfaces structurées contenant des champs et des contrôles :

```
Form {
    TextField(
        "Nom",
        text: $nom
    )
    TextField(
        "Courriel",
        text: $courriel
    )
}
```

Form offre automatiquement :

- Espacement et organisation
- Apparence cohérente avec le système

- Défilement
- Adaptation au système

Il est particulièrement utile pour les formulaires, les paramètres, les préférences et l'édition de données.

14. Section

Un formulaire peut être divisé en sections :

```
Form {
    Section("Informations personnelles") {
        TextField(
            "Nom",
            text: $nom
        )
        TextField(
            "Courriel",
            text: $courriel
        )
    }

    Section("Compte") {
        SecureField(
            "Mot de passe",
            text: $motDePasse
        )
    }
}
```

15. En-tête et pied de section

```
Section {
    SecureField(
        "Mot de passe",
        text: $motDePasse
    )
} header: {
    Text("Sécurité")
} footer: {
    Text("Le mot de passe doit être suffisamment sécurisé.")
}
```

L'en-tête (header) donne un titre à la section, et le pied (footer) ajoute une explication.

16. Exemple - Création d'un ticket

```
struct NouveauTicketView: View {
    @State private var titre = ""
    @State private var description = ""

    var body: some View {
        Form {
            Section("Demande") {
                TextField(
                    "Titre",
                    text: $titre
                )
                TextEditor(
                    text: $description
                )
                .frame(minHeight: 120)
            }
        }
        .navigationTitle("Nouveau ticket")
    }
}
```

navigationTitle s'affiche lorsque la vue se trouve dans une NavigationStack.

17. Un bouton d'enregistrement dans le formulaire

```
Form {
    Section {
        TextField(
            "Titre",
            text: $titre
        )
    }

    Section {
        Button("Enregistrer") {
            print(titre)
        }
    }
}
```

Le bouton occupe sa propre section et s'affiche comme une ligne du formulaire.

18. Annuler et Enregistrer dans une feuille

Rappel : la barre d'outils, dismiss() et la présentation d'une feuille ont été vus avec les feuilles (sheet).

Pour un formulaire présenté dans une feuille, Annuler et Enregistrer se placent dans la barre d'outils :

```
struct NouveauTicketView: View {
    @Environment(\.dismiss) private var dismiss

    @State private var titre = ""
    @State private var description = ""

    var body: some View {
        Form {
            Section("Demande") {
                TextField(
                    "Titre",
                    text: $titre
                )
                TextEditor(
                    text: $description
                )
                .frame(minHeight: 120)
            }
        }
        .navigationTitle("Nouveau ticket")
        .navigationBarTitleDisplayMode(.inline)
        .toolbar {
            ToolbarItem(placement: .cancellationAction) {
                Button("Annuler") {
                    dismiss()
                }
            }
            ToolbarItem(placement: .confirmationAction) {
                Button("Enregistrer") {
                    print(titre)
                    print(description)
                    dismiss()
                }
            }
        }
    }
}
```

La vue qui présente la feuille place le formulaire dans une NavigationStack :

```
Button("Nouveau ticket") {
    afficherAjout = true
}
.sheet(isPresented: $afficherAjout) {
    NavigationStack {
        NouveauTicketView()
    }
}
```

19. submitLabel

On peut modifier le texte du bouton d'action du clavier :

```
TextField(
    "Courriel",
    text: $courriel
)
.submitLabel(.next)
```

Valeur	Utilisation typique
.next	Passer au champ suivant
.done	Terminer la saisie
.search	Lancer une recherche
.send	Envoyer un message
.go	Valider et continuer

Attention : submitLabel ne change que le texte du bouton. Pour passer réellement au champ suivant, il faut utiliser @FocusState (voir plus loin).

20. onSubmit

On peut réagir à l'appui sur le bouton d'action du clavier :

```
TextField(
    "Recherche",
    text: $recherche
)
.submitLabel(.search)
.onSubmit {
    rechercher()
}
```

21. Gérer le focus avec @FocusState

@FocusState permet de savoir quel champ est actif et de lui donner le focus :

```
struct NomView: View {
    @State private var nom = ""
    @FocusState private var champActif: Bool

    var body: some View {
        VStack {
            TextField(
                "Nom",
                text: $nom
            )
            .focused($champActif)

            Button("Modifier le nom") {
                champActif = true
            }
        }
    }
}
```

- .focused($champActif) associe le champ à la variable
- Appuyer sur le bouton met champActif à true : le clavier s'ouvre sur le champ

22. Plusieurs champs avec @FocusState

Avec plusieurs champs, on utilise une énumération :

```
enum Champ {
    case nom
    case courriel
}
```

La variable devient alors un optionnel de cette énumération :

```
@FocusState
private var champActif: Champ?
```

- nil signifie qu'aucun champ n'a le focus
- .focused($champActif, equals: .nom) : ce champ a le focus lorsque champActif vaut .nom

23. Exemple - Passer d'un champ à l'autre

```
struct ContactView: View {
    @State private var nom = ""
    @State private var courriel = ""
    @FocusState private var champActif: Champ?

    var body: some View {
        Form {
            TextField(
                "Nom",
                text: $nom
            )
            .focused($champActif, equals: .nom)
            .submitLabel(.next)
            .onSubmit {
                champActif = .courriel
            }

            TextField(
                "Courriel",
                text: $courriel
            )
            .focused($champActif, equals: .courriel)
            .submitLabel(.done)
            .onSubmit {
                champActif = nil
            }

            Button("Terminer") {
                champActif = nil
            }
        }
    }
}
```

- Le bouton d'action du premier champ (.next) donne le focus au courriel
- Celui du deuxième champ (.done) ferme le clavier : champActif = nil
- Le bouton Terminer ferme aussi le clavier de la même façon

24. Champs numériques

SwiftUI peut lier un champ directement à une valeur numérique :

```
@State private var quantite = 0
@State private var prix = 0.0

var body: some View {
    Form {
        TextField(
            "Quantité",
            value: $quantite,
            format: .number
        )
        .keyboardType(.numberPad)

        TextField(
            "Prix",
            value: $prix,
            format: .number
        )
        .keyboardType(.decimalPad)
    }
}
```

Le format .number convertit le texte saisi en nombre. Cela évite de conserver temporairement toutes les valeurs sous forme de String.

25. Liaison avec un modèle

Une vue de modification peut recevoir le modèle avec @Binding :

```
struct Profil {
    var nom: String
    var courriel: String
}

struct ModifierProfilView: View {
    @Binding var profil: Profil

    var body: some View {
        Form {
            TextField(
                "Nom",
                text: $profil.nom
            )
            TextField(
                "Courriel",
                text: $profil.courriel
            )
        }
    }
}
```

$profil.nom donne un Binding<String> vers la propriété nom du modèle.

26. Attention : la modification est immédiate

Avec @Binding var profil: Profil, chaque modification du formulaire modifie immédiatement le modèle original.

Attention : si l'utilisateur appuie ensuite sur Annuler, les changements sont déjà appliqués.

27. Utiliser une copie temporaire

Pour un formulaire de modification, une meilleure approche consiste à travailler sur une copie, puis à l'appliquer seulement lors de Enregistrer :

```
struct ModifierProfilView: View {
    @Binding var profil: Profil
    @Environment(\.dismiss) private var dismiss

    @State private var nom: String
    @State private var courriel: String

    init(profil: Binding<Profil>) {
        _profil = profil
        _nom = State(initialValue: profil.wrappedValue.nom)
        _courriel = State(initialValue: profil.wrappedValue.courriel)
    }

    var body: some View {
        Form {
            TextField(
                "Nom",
                text: $nom
            )
            TextField(
                "Courriel",
                text: $courriel
            )
        }
        .toolbar {
            ToolbarItem(placement: .cancellationAction) {
                Button("Annuler") {
                    dismiss()
                }
            }
            ToolbarItem(placement: .confirmationAction) {
                Button("Enregistrer") {
                    profil.nom = nom
                    profil.courriel = courriel
                    dismiss()
                }
            }
        }
    }
}
```

- L'initialiseur copie les valeurs du modèle dans les @State
- Annuler ne modifie pas les données originales
- Enregistrer applique les modifications, puis ferme la feuille

28. Formulaire et ViewModel

Avec MVVM, la vue gère la saisie et l'affichage, le ViewModel la logique et le modèle les données. Le patron @Observable avec @Bindable a été vu dans la présentation sur MVVM :

```
@Observable
class ProfilViewModel {
    var nom = ""
    var courriel = ""

    func enregistrer() {
        // logique d'enregistrement
    }
}

struct ProfilView: View {
    @State private var viewModel = ProfilViewModel()

    var body: some View {
        @Bindable var viewModel = viewModel

        Form {
            TextField(
                "Nom",
                text: $viewModel.nom
            )
            TextField(
                "Courriel",
                text: $viewModel.courriel
            )
        }
    }
}
```

29. Résumé

Un formulaire doit être clair, court, bien organisé, facile à comprendre et adapté au type de données.

Besoin	Outil
Texte court	`TextField`
Mot de passe	`SecureField`
Texte long	`TextEditor`
Clavier adapté	`keyboardType`
Organisation	`Form et Section`

À éviter : mettre tous les champs dans un grand VStack sans organisation lorsque le formulaire devient complexe. Préférer Form et Section.

Catégorie	Outils
Contrôles	TextField, SecureField, TextEditor
Organisation	Form, Section
Expérience de saisie	.keyboardType(), .textInputAutocapitalization(), .autocorrectionDisabled(), .submitLabel(), .onSubmit(), @FocusState
Données	@State, @Binding, @Observable, @Bindable

31. Exercice - Ajouter un contact

1. Formulaire

Créer une vue :

```
AjouterContactView
```

contenant un formulaire avec :

- Nom
- Courriel
- Téléphone
- Notes

Utiliser TextField, TextEditor, Form et Section. Le champ Courriel doit utiliser le clavier approprié, et le champ Téléphone doit utiliser :

```
.phonePad
```

2. Barre d'outils

Ajouter une barre d'outils contenant Annuler et Enregistrer. Utiliser :

```
dismiss()
```

pour fermer le formulaire.

32. Exercice - Navigation au clavier

Ajouter :

```
@FocusState
```

afin que les champs Nom, Courriel et Téléphone puissent être parcourus, dans cet ordre, avec le bouton Suivant du clavier. Le dernier champ doit fermer le clavier.